Three-Layer Neural Network from Scratch

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

(X, y), _ = mnist.load_data()
X = X.reshape(-1, 784) / 255.0
Y = np.eye(10)[y]

W1 = np.random.randn(784, 64) * 0.01
W2 = np.random.randn(64, 10) * 0.01
b1, b2 = np.zeros((1, 64)), np.zeros((1, 10))

sigmoid = lambda x: 1 / (1 + np.exp(-np.clip(x, -500, 500)))
losses = []

for epoch in range(5):
    for i in range(0, 60000, 100):
        x, t = X[i:i+100], Y[i:i+100]
        a1 = sigmoid(x @ W1 + b1)
        a2 = sigmoid(a1 @ W2 + b2)

        d2 = (a2 - t) / len(x)
        d1 = (d2 @ W2.T) * a1 * (1 - a1)

        W2 -= 0.1 * (a1.T @ d2)
        b2 -= 0.1 * d2.sum(axis=0, keepdims=True)
        W1 -= 0.1 * (x.T @ d1)
        b1 -= 0.1 * d1.sum(axis=0, keepdims=True)

    loss = -np.mean(Y * np.log(
        sigmoid(sigmoid(X @ W1 + b1) @ W2 + b2) + 1e-8
    ))
    losses.append(loss)
    print("Epoch", epoch + 1, "Loss:", round(loss, 4))

plt.plot(losses)
plt.title("Training Loss")
plt.xlabel("Epoch")
```

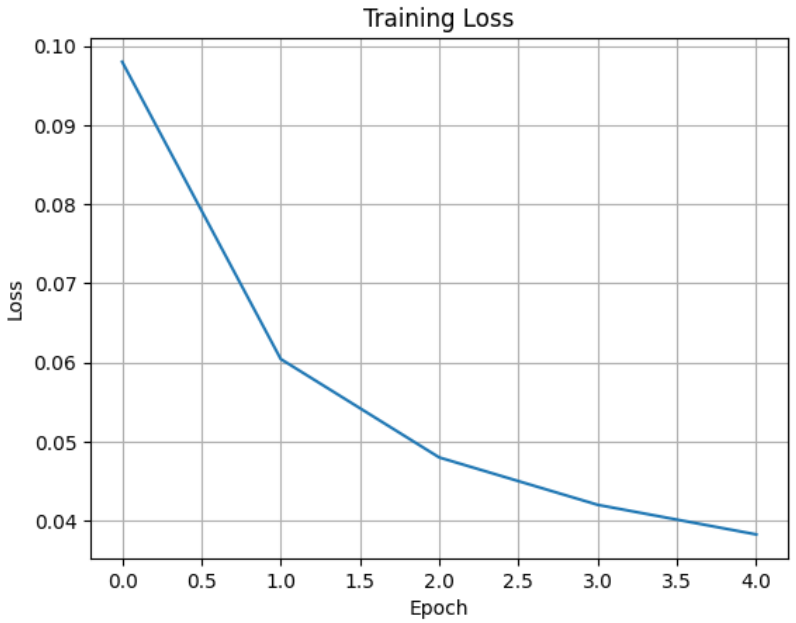
```
plt.ylabel("Loss")
plt.grid()
plt.show()

i = 100
img = X[i].reshape(1, 784)
pred = np.argmax(sigmoid(sigmoid(img @ W1 + b1) @ W2 + b2))

plt.imshow(X[i].reshape(28, 28), cmap="gray")
plt.title(f"Actual: {y[i]}, Predicted: {pred}")
plt.axis("off")
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1 Loss: 0.098
Epoch 2 Loss: 0.0604
Epoch 3 Loss: 0.048
Epoch 4 Loss: 0.042
Epoch 5 Loss: 0.0382
```



Actual: 5, Predicted: 5

